



AYUDANTÍA I2

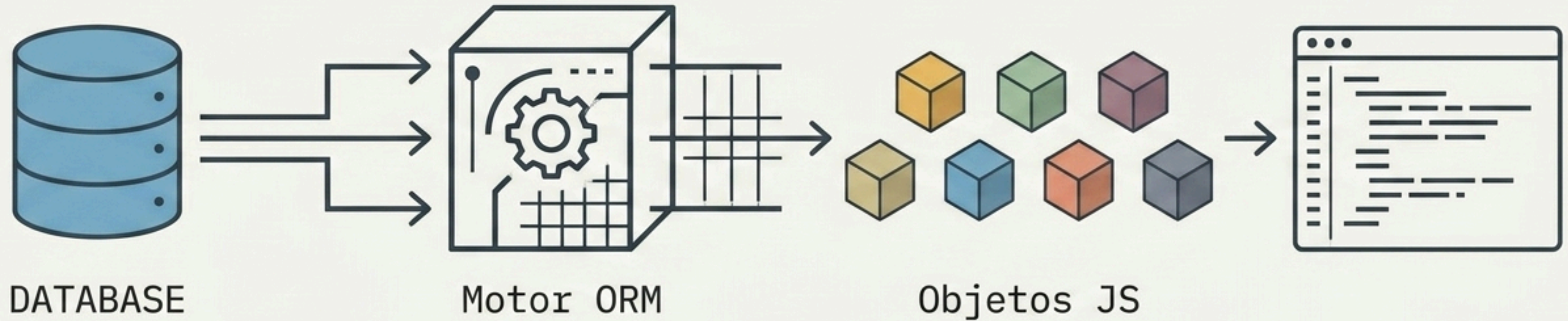
JUAN JOSÉ MERUANE
ARTURO PACHECO



RECORDATORIO AYUDANTIA PASADA



Object Relational Mapping (ORM)



Mapea estructuras de una tabla relacional a entidades lógicas.

Interacción directa mediante métodos, sin escribir SQL a mano.

¿Por qué necesitamos un ORM?

SQL Tradicional

```
SELECT * FROM usuarios WHERE id = 1
```

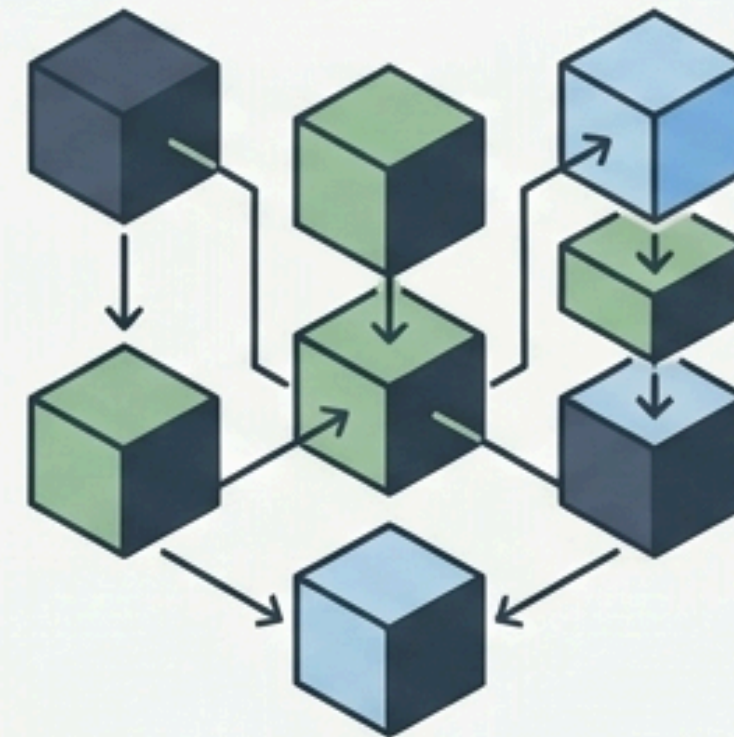
Mezclar strings en JS es desordenado, inseguro y difícil de mantener.



ORM Moderno

```
await Usuario.findAll()
```

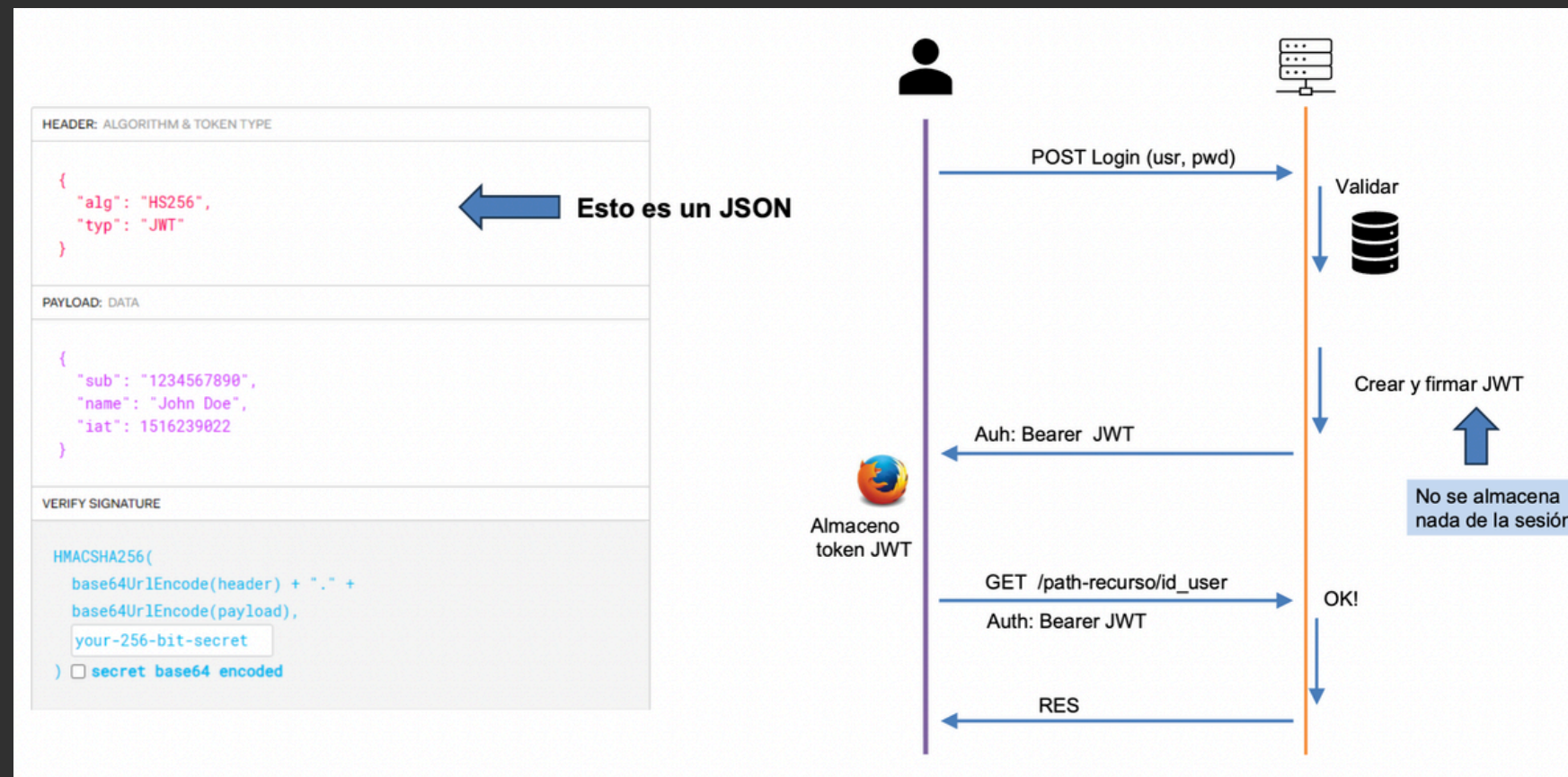
Objetos nativos, autocompletado, código escalable e intuitivo.



SEGURIDAD JWT



JWT (JSON Web Tokens): Es un estándar abierto de la RFC 7519 que describe un método para estructurar intercambio seguro de "tokens" entre dos interlocutores (sistemas).



- El usuario se conecta al sitio con su user y password.
- El servidor autentifica al usuario y le entrega un JWT firmado con la firma secreta que conoce el server (llave privada).
- El cliente (lado usuario) usa el JWT entregado, para acceder a recursos protegidos.
- Cada vez que se accede a los recursos protegidos, el servidor verifica la autenticidad del token entregado, usando para ello la llave privada.

SEGURIDAD BCRYPT/HASHING



Por **seguridad** cierta información sensible se **DEBE** gestionada, controlada y/o guardada de manera **segura**, por ejemplo las contraseñas

bcrypt usa un **algoritmo de hashing** diseñado específicamente para almacenar contraseñas de forma segura.

El hash es unidireccional, no se puede "desencriptar". Al autenticar, se compara la contraseña ingresada con el hash guardado usando un algoritmo que produce siempre el mismo hash para la misma dupla (contraseña, salt)

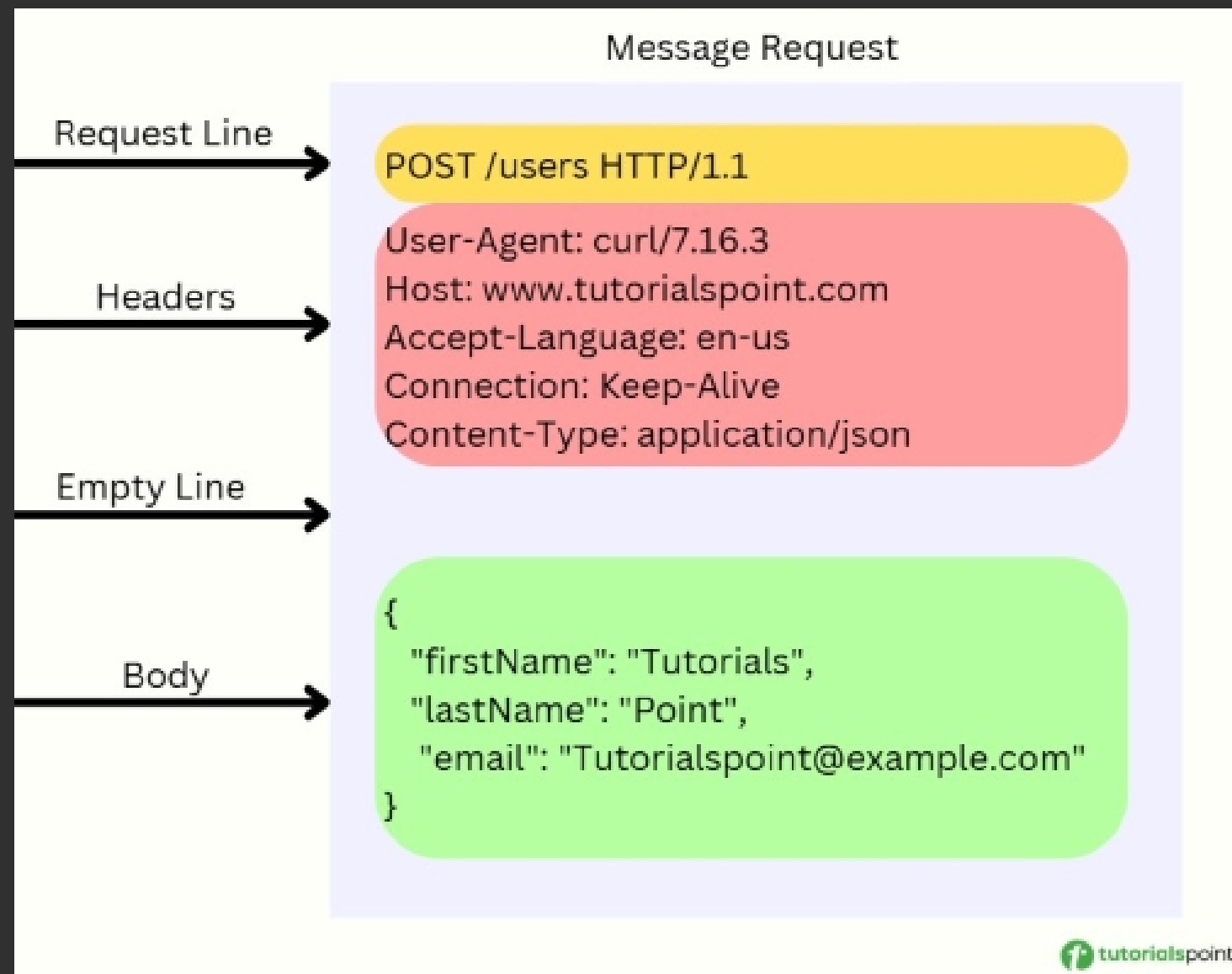
1. **Salt Generation:** genera un "salt", que es un valor aleatorio único que se añade a la clave/contraseña.

1. **Hashing:** se combina la clave + salt y se aplica un proceso repetido de hashing, por defecto 10 veces, configurable



HTTP REQUEST

Métodos HTTP (similar a CRUD)



Método HTTP → qué acción se quiere hacer

URI/Path → recurso solicitado

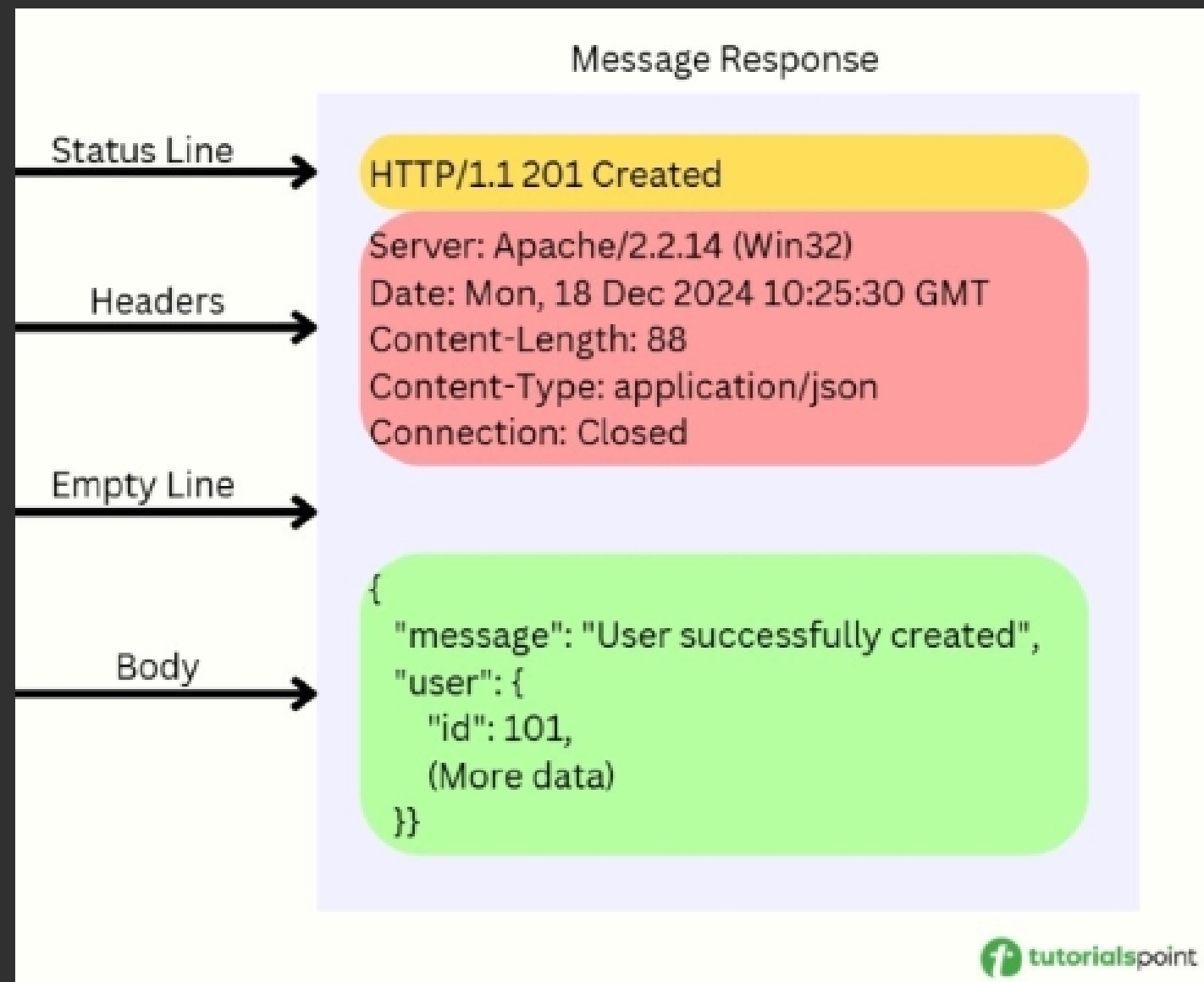
Versión HTTP → ej. HTTP/1.1

Headers → metadata (cookies, content-type, auth, etc.)

Body → datos enviados (opcional)

Método	Función	CRUD
GET	Obtener datos	Read
POST	Enviar/crear datos	Create
PUT	Reemplazar recurso	Update
PATCH	Actualizar parcialmente	Update
DELETE	Eliminar	Delete
HEAD	Obtener headers	—
OPTIONS	Ver métodos disponibles	—
CONNECT	Conexiones seguras SSL/TLS	—

HTTP RESPONSE

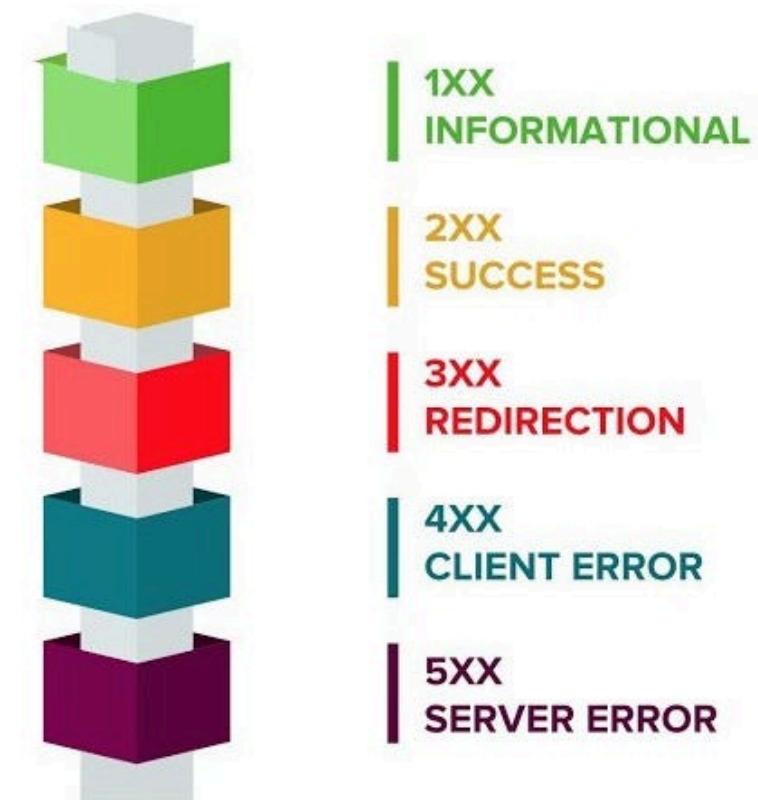


Línea de estado → versión + código + explicación

Headers → información extra

Body → recurso o datos solicitados

HTTP Status Codes



API REST

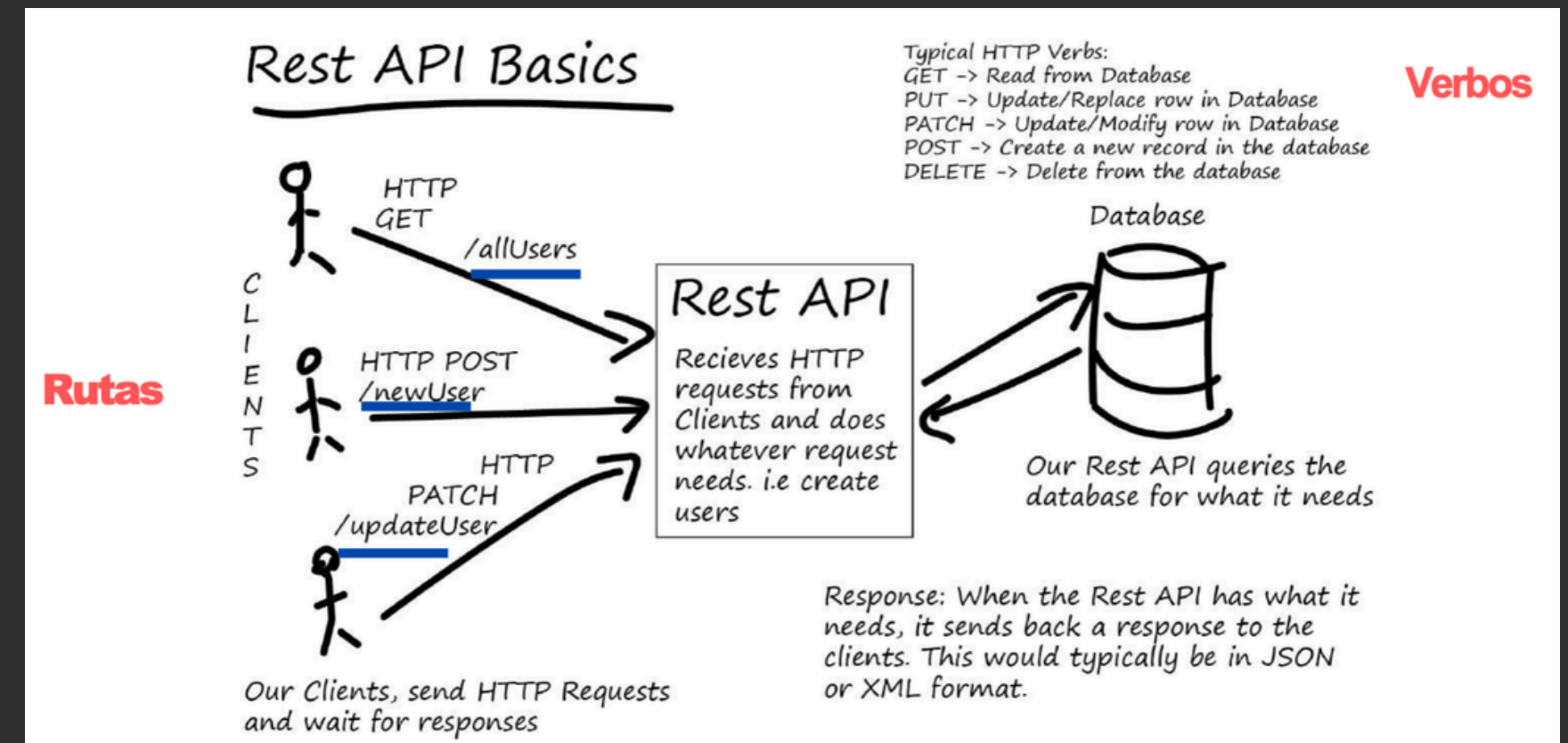
API debe definir **el protocolo de Comunicaciones**, las **reglas** que se deben seguir para comunicarse con ella y la **definición** de la **entrada** y la **salida** de la misma.

Una API REST se guía por los principios de:

- Ser **cliente-servidor**. Separa cliente (el browser y lo que se ve en él) y Servidor información y operaciones, bdd)
- **Stateless**

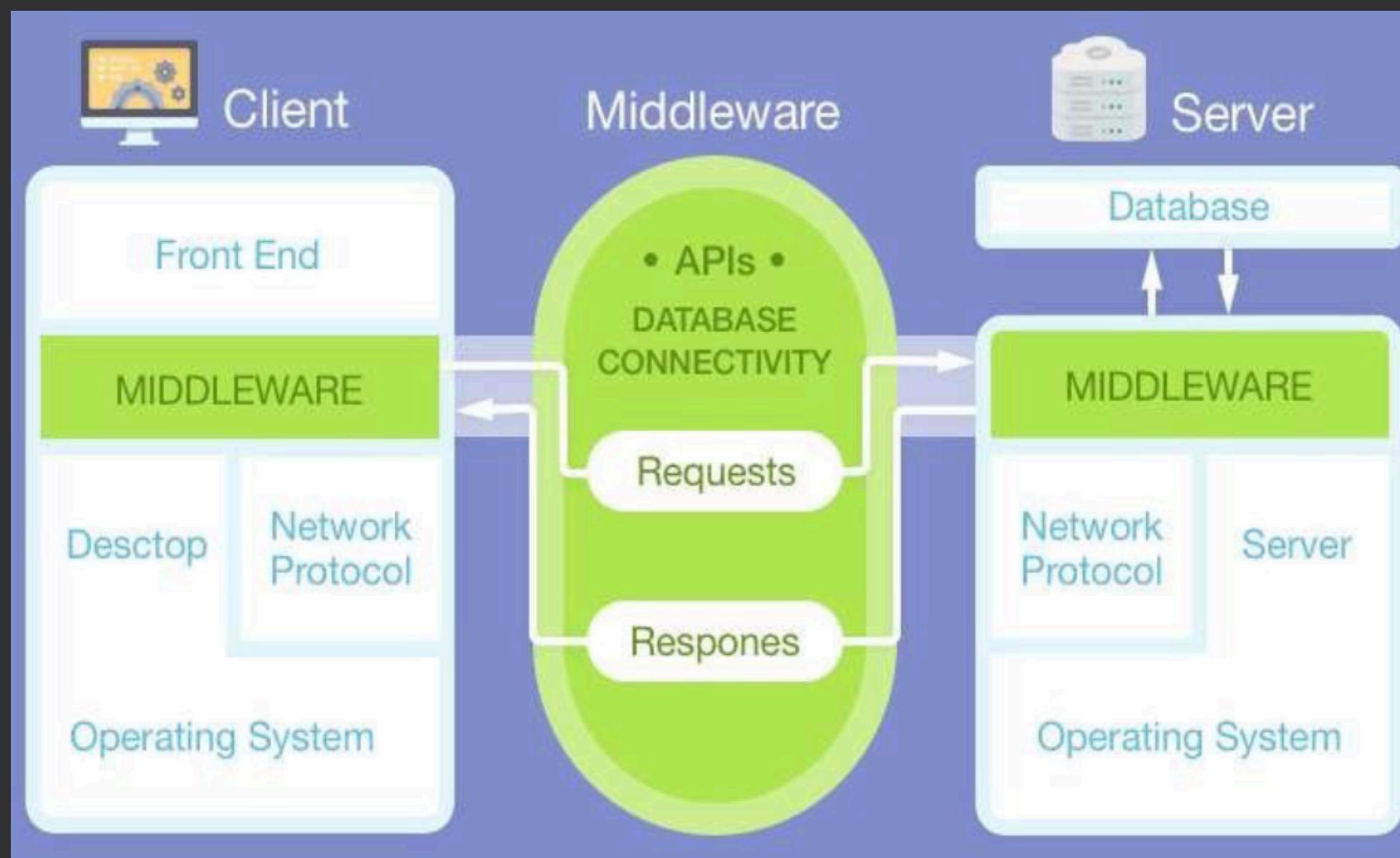
Y ES:

- Utiliza una interface uniforme
- Debe implementar "cache"
- arquitectura debe ser pensada, diseñada y/o implementada en capas (el cliente solo la primera)



Recordar Documentación con HTTP, REQ, RESP, ERROR
y EJEMPLOS
(como hicieron en el proyecto)

MIDDLEWARE Y KOA



El middleware es una pieza de software que reside entre la capa cliente (lado del usuario) y el servidor.

Controles de flujo típicos para un middleware son la autenticación, validación, almacenamiento de transacciones, etc.

En lo que se conoce como ciclo de requerimiento-respuesta (request-response) donde, para atender todos los requerimientos de un cliente, se encadenan requests en capas encadenadas

MIDDLEWARE Y KOA

En Koa, el objeto de aplicación (app) contiene la instancia de la app y una lista (stack) de middlewares, que se ejecutan en cascada cuando llega una solicitud.

El contexto (ctx) combina los objetos de request y response de Node en un solo objeto, creado para cada petición.

Dentro de ctx destacan:

- ctx.request: maneja información del request (petición).
- ctx.response: maneja la respuesta HTTP.
- ctx.state: sirve para compartir datos entre middlewares.

Los middlewares en Koa son funciones asíncronas que reciben dos argumentos:

- ctx (contexto)
- next (ejecuta el siguiente middleware)

Se registran con:

```
app.use(miMiddleware);
```

ARQUITECTURA DE APLICACIÓN KOA.JS: GESTIÓN DE MIDDLEWARE EN CASCADA



COMPONENTES CLAVE



EL OBJETO DE APLICACIÓN:

- Core Koa instance.
- Contiene array ordenado de middlewares.
- Gestiona la ejecución en cascada.



EL CONTEXTO KOA (ctx):

- Objeto único por request.
- Combinación de Node's Request & Response.
- Acceso: 1er argumento del middleware.

🌐 ctx.request (Instancia de request de Koa)

💬 ctx.response (Instancia de response de Koa)

📁 ctx.state (Recomendado para pasar datos)

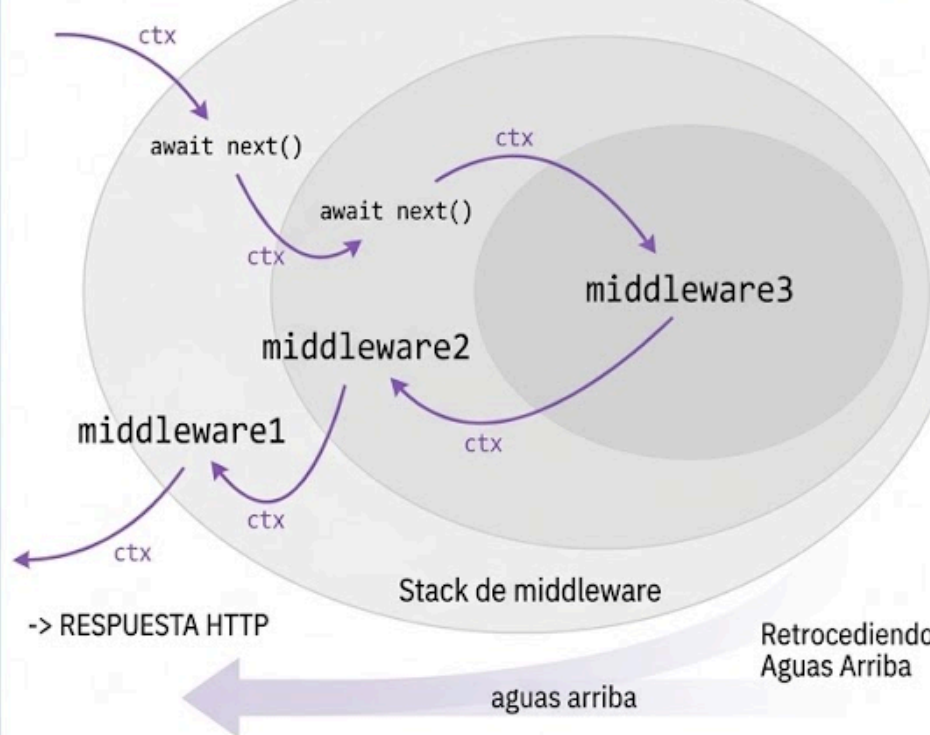


MIDDLEWARE & FLUJO:

- Funciones async (ctx, next).
- Registradas con app.use(funcionRef).
- Llamar a next() suspende y pasa control downstream.
- Se desenrolla y reanuda upstream al finalizar.

PETICIÓN HTTP →

aguas abajo



Cuando un middleware ejecuta next(), cede el control al siguiente middleware. Al terminar todos, la ejecución vuelve hacia atrás (“desenrollado”), permitiendo seguir ejecutando código después de next().

EJERCICIOS

COOKIES, SESSION, JWT, OAUTH

La universidad quiere modernizar el sistema de login de Siding. Actualmente usan sesiones de servidor con cookies y se quiere migrar a JWT y también está la idea de integrar OAuth para permitir login usando Google.

a) Explique las diferencias entre Cookies de sesión, Sesiones en servidor y JWT como mecanismo de autenticación. Incluya:

1. Dónde se guarda el estado
2. Qué viaja en cada request
3. Riesgos y mitigaciones (recuerde entre otras cosas: fechas, cabeceras, etc.)

b) Recomiende un diseño híbrido indicando cuándo usar sesiones basadas en cookies y cuándo JWT. Justifique según seguridad y escalabilidad.

COOKIES, SESSION, JWT, OAUTH

La universidad quiere modernizar el sistema de login de Siding. Actualmente usan sesiones de servidor con cookies y se quiere migrar a JWT y también está la idea de integrar OAuth para permitir login usando Google.

a) Explique las diferencias entre Cookies de sesión, Sesiones en servidor y JWT como mecanismo de autenticación.

Aspecto	Cookies + Session	JWT
Estado	Servidor	Dentro del token
Cliente guarda	Session ID	JWT completo
Request	Cookie con ID	Bearer Token
Escalabilidad	Más difícil	Más fácil
Logout	Fácil invalidar	Menos directo

COOKIES, SESSION, JWT, OAUTH

a) Explique las diferencias entre Cookies de sesión, Sesiones en servidor y JWT como mecanismo de autenticación. Incluya: **Dónde se guarda el estado. Qué viaja en cada request**

Cookies + Session

El navegador guarda solo un session ID.

El estado real está en el servidor.

Cada request envía la cookie y el servidor busca la sesión.

1. Login
2. Servidor crea sesión
3. BDD guarda sesión
4. Browser recibe cookie(sessionID)
5. Cada request envía cookie
6. Servidor busca sesión

JWT

La información viaja dentro del token.

Es stateless, no depende de sesión almacenada.

Cada request manda el JWT completo.

1. Login
2. Servidor genera JWT
3. Browser guarda token
4. Request: Authorization Bearer <token>
5. Servidor valida firma

COOKIES, SESSION, JWT, OAUTH

a) Explique las diferencias entre Cookies de sesión, Sesiones en servidor y JWT como mecanismo de autenticación. Incluya: Riesgos y mitigaciones (recuerde entre otras cosas: fechas, cabeceras, etc.)

Riesgos

Robo de cookies/tokens (XSS, robo de dispositivo, MITM sin HTTPS)

Tokens/ID sin expiración razonable

Almacenamiento inseguro
(localStorage en vez de sessionStorage)

Mitigaciones

Cookies:

HttpOnly, Secure, SameSite

JWT:

Validar la firma y aumentar rounds de salt
Uso de refresh tokens

General:

HTTPS
Expiración en cookies y JWT

COOKIES, SESSION, JWT, OAUTH

b) Recomiende un diseño híbrido indicando cuándo usar sesiones basadas en cookies y cuándo JWT. Justifique según seguridad y escalabilidad.

Sesiones + Cookies

Web tradicional (browser + backend)

Paneles de administrador

Áreas sensibles

Cuando se necesita control de sesión

Uso de HttpOnly y SameSite

Escalabilidad

Requieren almacenamiento compartido

Ej: BDD o Redis entre servidores

Seguridad

El servidor controla la sesión

Fácil invalidar en logout

Mayor control sobre usuarios activos

JWT

APIs REST externas

Comunicación entre microservicios

Sistemas distribuidos

Cuando importa la escalabilidad

Arquitectura stateless

Escalabilidad

Stateless

Solo verificar firma

Menor carga en el servidor

Seguridad

Más difícil revocar tokens

Se mitiga con:

expiración corta

refresh tokens

API REST, HTTP, CRUD, CODIGOS DE ESTADO

Proponga el diseño de rutas y verbos HTTP correctos (o sea endpoints) para:

1. Crear un curso
2. Listar todos los cursos
3. Editar el nombre del curso
4. Eliminar un curso

API REST, HTTP, CRUD, CODIGOS DE ESTADO

1. Crear curso: POST /cursos

2. Listar todos los cursos: GET /cursos

3. Editar nombre del curso

PUT /cursos/{id} (curso completo)

PATCH /cursos/{id} (solo campo nombre) Ambas opciones son validas

4. Eliminar curso: DELETE /cursos/{id}

API REST, HTTP, CRUD, CODIGOS DE ESTADO

Dado el siguiente request de un alumno:

GET /cursos/999

El curso no existe. Como diseñador de la API, ¿qué status code (código) retornaría y qué estructura JSON mínima sugeriría? (si no sabe el código exacto, al menos indique el rango numérico del error). Justifique brevemente la elección de dicho código.

API REST, HTTP, CRUD, CODIGOS DE ESTADO

404 (Not Found)

```
{  
  "error": "Curso no encontrado",  
  "code": "COURSE_NOT_FOUND"  
}  
  
O  
  
{  
  "message": "Curso 999 no existe"  
}
```

BCRYPT, POLÍTICA DE CONTRASEÑAS

El equipo quiere asegurar que los estudiantes creen contraseñas robustas en su nueva plataforma.

Explique:

1. Qué es bcrypt
2. Para qué sirve el salt
3. Qué significa salt rounds y su impacto en seguridad y desempeño
4. Por qué el hash es irreversible

Diseña una política de contraseñas adecuada para estudiantes (longitud, caracteres especiales, evitar información personal, etc.)

BCRYPT, POLÍTICA DE CONTRASEÑAS

1. Qué es bcrypt

bcrypt es una función/biblioteca de hashing seguro para contraseñas. Se utiliza para almacenar contraseñas de forma segura en bases de datos, transformándolas en un hash irreversible en lugar de guardar la contraseña original.

Características principales:

- Usa salt automático para evitar hashes repetidos.
- Permite configurar un factor de costo (salt rounds).
- Está diseñado para ser lento, dificultando ataques de fuerza bruta.

Importante: bcrypt no cifra contraseñas, las hashea.

2. ¿Para qué sirve el salt?

El salt es un valor aleatorio que se agrega a la contraseña antes de generar el hash. Sirve para:

- Evitar que dos contraseñas iguales tengan el mismo hash.
- Prevenir ataques con tablas rainbow, donde atacantes usan hashes precalculados.
- Evitar comparar hashes entre usuarios para detectar contraseñas iguales.

Ejemplo:

Contraseña: "hola123"

Usuario A → salt distinto → hash A

Usuario B → salt distinto → hash B

Aunque ambos usuarios tengan la misma contraseña, los hashes serán diferentes. En bcrypt, el salt viene incorporado en el resultado final del hash.

BCRYPT, POLÍTICA DE CONTRASEÑAS

3. ¿Qué significa salt rounds y cuál es su impacto?

Los salt rounds (o cost factor) representan el nivel de costo computacional del algoritmo, es decir, cuántas iteraciones/procesamiento se aplican para generar el hash.

Impacto en seguridad

- Más rounds = más seguridad
- Hace más lento probar millones de contraseñas, dificultando ataques de fuerza bruta.

Impacto en rendimiento

- Más rounds = más tiempo de procesamiento
- Puede aumentar el consumo del servidor y hacer el login más lento si se configura demasiado alto.

Ejemplo:

- 10 rounds → rápido, seguridad aceptable.
- 12–14 rounds → más seguro, pero más costoso computacionalmente.

El objetivo es encontrar un equilibrio entre seguridad y desempeño.

BCRYPT, POLÍTICA DE CONTRASEÑAS

4. ¿Por qué el hash es irreversible?

El hash es irreversible por diseño porque utiliza una función unidireccional, es decir, transforma datos de entrada en un resultado sin existir una operación matemática inversa para recuperar el original.

Diferencia con cifrado:

- Cifrado: se puede revertir usando una clave.
- Hash: no existe una clave para “deshashear” el valor.

Por eso, cuando un usuario inicia sesión, el sistema no recupera la contraseña original, sino que hashea la contraseña ingresada y compara los hashes.

BCRYPT, POLÍTICA DE CONTRASEÑAS

5. Política de contraseñas para estudiantes

Política propuesta

1. Longitud mínima: Mínimo 10–12 caracteres, ya que contraseñas largas son más difíciles de adivinar.
2. Variedad de caracteres: Combinar mayúsculas, minúsculas, números y símbolos especiales para aumentar complejidad.
3. Evitar información personal: No usar nombre, apellido, RUT, fecha de nacimiento, mascota o carrera, ya que son fáciles de obtener.
4. Bloquear contraseñas comunes: Prohibir claves débiles como:
 - 123456, password, clave123, qwerty
5. No reutilizar contraseñas anteriores: Evita que filtraciones antiguas comprometan la cuenta.
6. Protección ante intentos fallidos: Aplicar bloqueo temporal o retardo después de varios intentos incorrectos para reducir ataques de fuerza bruta.
7. Verificación de fortaleza: Usar herramientas como zxcvbn para evaluar si la contraseña es segura.

Ejemplo de contraseña segura:

M1Universidad!2026

Ejemplo de contraseña insegura:

Juan2004 o Ingenieria123



AYUDANTÍA I2

JUAN JOSÉ MERUANE
ARTURO PACHECO